

Evo-Kernel

个人经验治理与固化层

使用手册

zodyne

2026 年 7 月 29 日

摘要

Evo-Kernel 是位于通用记忆框架之上的「经验治理与固化层」。它以纯文件 + git 作为单一事实源，通过捕获、蒸馏、人审、固化四个环节，把零散的工作经验转化为可检索、可审计、可硬化的约束与技能。

本手册面向日常使用者，覆盖：快速开始、核心 workflow、周期维护、备份恢复、以及使用中的红线与注意事项。

目录

1	Evo-Kernel 是什么	3
1.1	一句话定位	3
1.2	核心能力	3
1.3	架构分层	3
2	目录与状态机	4
2.1	目录角色	4
2.2	经验生命周期	4
3	快速开始	5
3.1	日常检查命令	5
3.2	捕获一个想法	5
3.3	主动召回经验	6
4	核心 workflow	6
4.1	workflow 一：从 capture 到 curate	6
4.2	workflow 二：会话蒸馏	7
4.3	workflow 三：技能固化	8
4.4	workflow 四：批量导入外部资料与技术资料	9
4.4.1	导入的两阶段模型	9
4.4.2	技术资料进哪个区	10

4.4.3	示例：从 Markdown 仓库批量导入	10
4.4.4	研究与借鉴	11
4.5	workflow五：硬约束硬化	12
5	周期维护	12
5.1	每周/每两周做一次	12
5.2	对账与计数	13
5.3	治理动作	13
6	硬约束执行原理	14
6.1	guard 与 hook-guard	14
7	备份与恢复	14
7.1	备份机制	14
7.2	恢复演练	15
8	红线与注意事项	15
8.1	常见误区	16
9	实施进度评估（截至 2026-07-29）	16
9.1	总体结论	16
9.2	Phase 0 清单核销	17
9.3	能力落地矩阵	18
9.4	库存与飞轮实测数据	19
9.5	判据门控状态	19
9.6	尚未验证的路径（诚实清单）	20
9.7	建议的下一步（按优先级）	20
10	命令速查表	21

1 Evo-Kernel 是什么

1.1 一句话定位

通用记忆框架 (Mem0 / Zep / Letta 等) 解决的是经验的「存储与检索」; Evo-Kernel 解决的是其上没人解决的「治理、固化、可审计」问题。二者是叠加而非替代。

1.2 核心能力

- **捕获**: 零判断入口, 任何想法先丢进 inbox。
- **检索注入**: 按任务自动从 playbook / facts / episodes / principles 召回相关经验。
- **离线蒸馏**: session 结束后对 transcript 切片, 由 Reflector 生成提案, 人审后入库。
- **技能固化**: 经验 → SKILL.md, 可直接被 Claude Code / Pi 挂载。
- **硬约束硬化**: 经验 → guard / hook, 对危险工具调用进行 warn / block。
- **git 可审计**: 每一次 curate / solidify / demote 都是一次可 diff、可 revert 的 commit。

1.3 架构分层

图 1 展示了 Evo-Kernel 的层次结构。上层是 harness (Claude Code / Pi), 下层是纯文件 + git 的内核, 中间通过 hooks 与 CLI 交互。

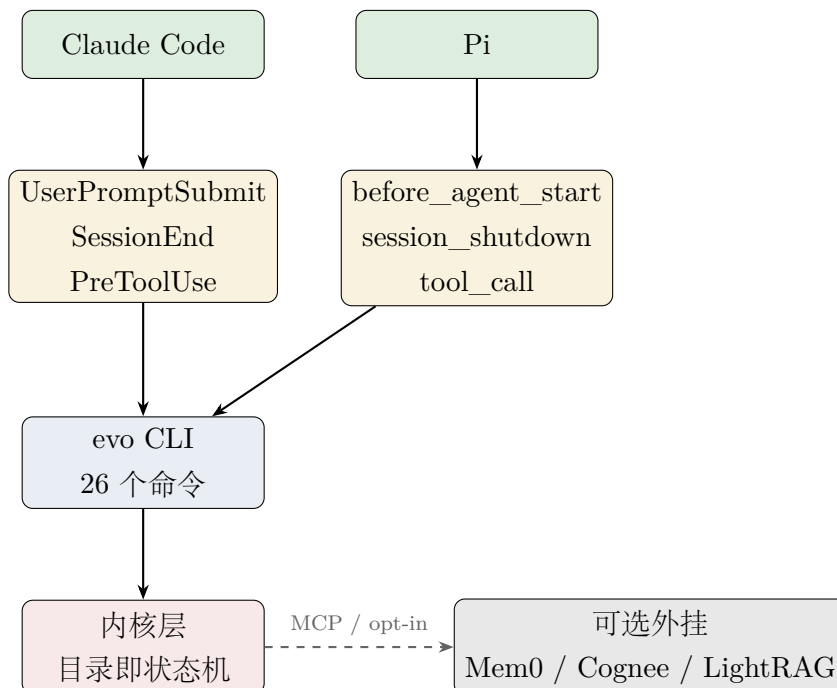


图 1: Evo-Kernel 架构分层

2 目录与状态机

2.1 目录角色

Evo-Kernel 的每个目录都有明确的角色，这是「目录即状态机」的体现：文件在不同目录间移动，即代表经验状态的变化。

表 1: 目录角色与注入权限

目录	角色	是否注入
inbox/	capture 暂存 + 会话登记	否
lessons/	candidate 经验暂存	否
facts/	事实/偏好/环境	是
episodes/	情景记忆：一次任务一份	是
playbook/	策略库，带 helpful/harmful 计数	是
principles/	跨域普适原则	是
skills/	SKILL.md 包 (agentskills.io)	经 skill 注入
ops/archive/	退役/固化存档	否
ops/constraints/	硬约束规则 (JSON)	guard 执行
ops/log/	运行日志 (全部 gitignore)	—
ops/proposals/	reflect/distill 提案	待人审
index/manifest.yaml	派生索引 (可重建)	—

红线

不变量 I2: inbox/ 与 lessons/ 中的条目永远不会被自动注入 context。只有经过人审 curate 进入 playbook / facts / episodes / principles 的条目，才有资格被 recall。否则 capture 这个零审查入口就成了绕过人审的后门。

2.2 经验生命周期

图 2 展示了一条经验从捕获到固化或退役的完整路径。

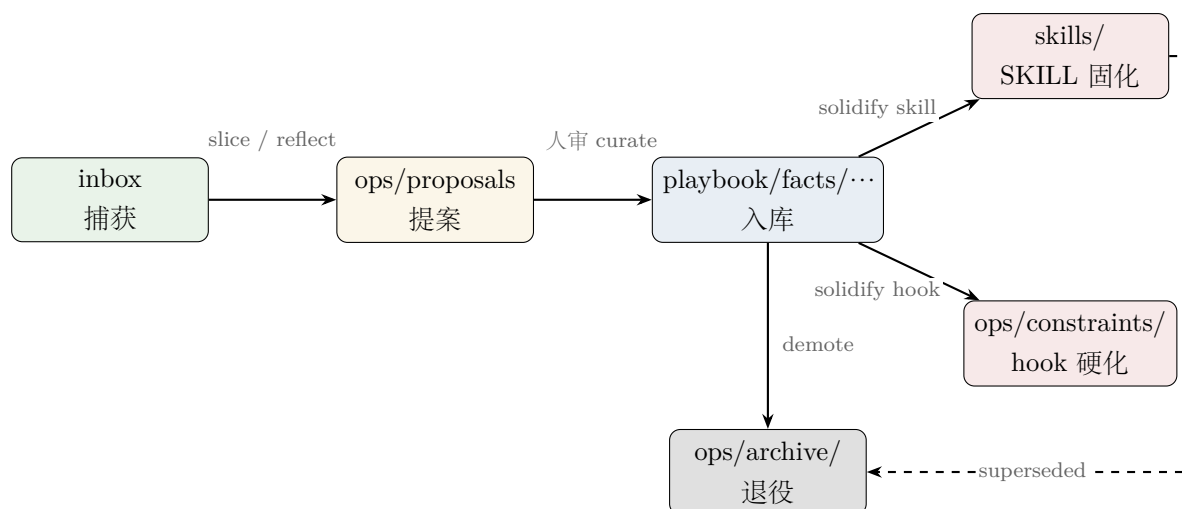


图 2: 经验生命周期状态机

3 快速开始

3.1 日常检查命令

每天开始工作前，先检查系统健康：

```
cd ~/Dev/evo-kernel
./bin/evo doctor
```

期望输出是 OK (N PASS / 少量 WARN / 0 FAIL)，只要 FAIL=0 即为健康。常见的 WARN 有三类，均非故障：工作区有未提交变更、manifest 滞后（跑一次 `evo index rebuild` 即消）、transcript 哨兵比（已知飞轮状态，由 reflect 周期跟踪）。

3.2 捕获一个想法

任何时候想到一个值得记录的经验、坑或偏好，直接 capture：

```
./bin/evo capture "遇到 Claude Code 权限拦截时，先用
AskUserQuestion 或明确授权，不要偷偷绕过"
```

这会在 `inbox/` 下生成一个带时间戳的 markdown 文件。capture 是零判断入口，不需要结构化，不需要 frontmatter。

提示

capture 的目标不是写完美条目，而是不错过任何一闪而过的经验。结构化和人审留给后面的 curate。

3.3 主动召回经验

如果你在写代码或调试，想主动看看有没有相关经验：

```
./bin/evo recall --task "如何安全地删除远程 git 分支"
```

这会输出一段可注入的文本（如果命中）。更常见的场景是 Claude Code / Pi 自动在每次输入时调用 `hook-recall`，你不需要手动跑。

4 核心 workflow

4.1 工作流一：从 capture 到 curate

这是最常见的单条经验入库路径。

Step 1: 把 inbox 里的 raw 文本整理成提案

```
./bin/evo inbox
# 看到一条 capture，打开编辑
# 把它改写成符合 SCHEMA.md 的提案文件，放到 ops/proposals/
```

一个最小提案示例：

```
---
id: avoid-silent-bypass
name: 权限拦截不绕过
type: principle
status: candidate
scope: [workflow]
domains: [claude-code]
triggers:
  - 权限拦截
  - Claude Code 拒绝
  - bypass
evidence:
  helpful: 0
  harmful: 0
verified_by: human
---
```

当 Claude Code 的权限系统拦截一个操作（如删除远程分支、修改系统配置）时，

正确动作是：

1. 用 `AskUserQuestion` 向用户明确请求授权；

2. 或把命令贴给用户，让用户用 `!` 前缀自己执行。

错误动作是：

- 用其他工具绕过权限拦截；
- 把破坏性操作伪装成无害命令。

Step 2: 人审并入库

```
./bin/evo curate --file ops/proposals/avoid-silent-bypass.md --to
principles
```

curate 会做三件事：

1. schema 校验 (id / type / status / triggers 必填)。
2. 敏感信息检查 (凭据、密钥、内网地址、PII 不得入库)。
3. 指令样内容审查 (正文必须是陈述式经验，不能是面向未来 agent 的祈使指令)。

通过后，文件被移动到 `principles/`，自动 rebuild manifest、commit、并 push 到 remote。

注意

curate 是唯一的入库口 (不变量 I3)。不要手动把文件拖进 `playbook/` 或 `facts/`，否则绕过了敏感信息和指令样审查。

4.2 工作流二：会话蒸馏

一次完整工作会话结束后，把 transcript 中的经验提炼出来。

提示

这条流程现在已经自动化：SessionEnd 触发器会调用 `ops/bin/evo-distill.sh` (带锁，防并发)，launchd 每日 03:30 再跑一次兜底 (`--max 3`)，清掉会话结束时机器休眠、harness 超时或锁冲突漏掉的积压。队列由 `evo queue` 给出、完成后 `evo mark-distilled` 回写。下面的手动步骤保留，用于自动链路失效时的排查与补跑。

Step 1: 确认 session 已登记

Claude Code / Pi 的 session-end hook 会自动写 `inbox/session-refs.jsonl`。你也可以手动登记：

```
./bin/evo session-end --session /path/to/transcript.jsonl --id <
session-id>
```

Step 2: 切片

```
./bin/evo slice --session /path/to/transcript.jsonl --ids <
    session-id>
```

这会输出一个 Markdown 报告，包含：命令 ↔ 结果对照、写/改文件列表、首条 user、末条 assistant，以及该 session 被注入了哪些经验条目。

Step 3: 生成 reflect 提案

```
./bin/evo reflect --save
```

reflect 会：

- 统计库状态（按 zone、status 分桶）；
- 分析 recall.jsonl（调用数、命中数、空命中任务）；
- 跑 audit，列出治理问题；
- 输出判据对照表（M1/M2/P4 判据、transcript 时效、降级盘点等）；
- --save 时把报告写入 ops/proposals/reflect-< 日期 >.md。

Step 4: 人审并 curate

打开 ops/proposals/reflect-< 日期 >.md，把认可的经验改写成标准提案文件，然后：

```
./bin/evo curate --file ops/proposals/<proposal>.md --to <
    playbook|facts|episodes|principles>
```

4.3 workflow三：技能固化

当一条经验反复有用、值得变成可挂载的技能时：

```
./bin/evo solidify --id avoid-silent-bypass --to skill
```

这会：

1. 在 skills/avoid-silent-bypass/SKILL.md 创建技能包；
2. 把原条目标记为 solidified, 写入 superseded_by: skill:avoid-silent-bypass, 移到 ops/archive/;
3. 自动 rebuild manifest、调用 evo link 同步软链、commit、push。

之后，该 skill 会出现在 ~/.claude/skills/ 软链中，Claude Code 可以加载。

4.4 工作流四：批量导入外部资料与技术资料

如果你已经有大量技术笔记、文档摘录、会议记录或项目沉淀（例如从 Obsidian、Notion、Markdown 仓库导出），不要直接拖进 `facts/` 或 `playbook/`——那样会绕过 `curate` 的人审、敏感信息检查和指令样审查（违反不变量 I3）。

4.4.1 导入的两阶段模型

借鉴 Zettelkasten 的 “fleeting note → literature note → permanent note” 流程，以及 RAG ingestion pipeline 的 “extract → clean → chunk → enrich → index” 模式，Evo-Kernel 推荐采用 **机器预处理 + 人工审查** 的两阶段模型：

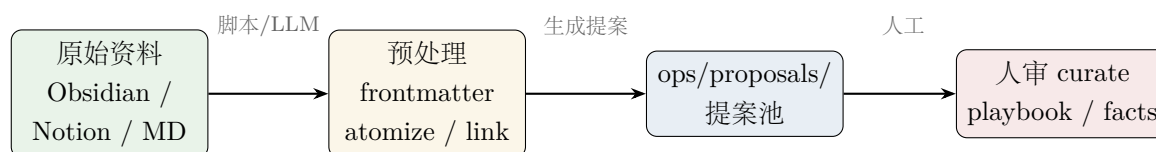


图 3: 外部资料导入的两阶段模型

第一阶段：机器预处理（可自动化）

- **格式规范化**：补全 YAML frontmatter (id、type、status、triggers、domains、scope)，统一文件名；
- **原子化**：把长文档拆成一条一条原子经验（一个知识点/一个检查清单/一个案例一条）；
- **链接转换**：把 wikilink、Notion link 转成 Evo-Kernel 的 id 引用或普通 Markdown link；
- **元数据提取**：用 LLM 或规则自动生成 triggers、domains、scope；
- **暂存提案池**：所有预处理结果放到 `ops/proposals/import-< 批次 >/`，不进注入通道。

第二阶段：人工审查入库（不可跳过）

- 逐条阅读提案，删除噪音、重复、过时的资料；
- 把认可的内容改写成 Evo-Kernel 的陈述式经验；
- 运行 `evo curate` 进入 `facts/`、`playbook/` 或 `episodes/`。

红线

为什么不要一轮直接导入？

- 1. 绕过 `curate` 的敏感信息检查，可能把凭据、内网地址、客户信息推入 `git`；
- 2. 绕过指令样审查，可能把祈使式指令（“总是执行 X”）固化成持久化提示注入；
- 3. 未经原子化的长文档会降低 `recall` 精度，大段无关文本稀释 `triggers`；
- 4. 噪音和重复条目直接进入 `playbook/`，会污染注入集、拉低 `helpful/harmful` 计数质量。

4.4.2 技术资料进哪个区

根据资料性质选择目标目录：

表 2: 技术资料的目标分区

目录	适合内容	示例
<code>facts/</code>	知识点、API 行为、环境配置	“Next.js 15 Server Actions 必须 <code>async</code> ”
<code>playbook/</code>	可复用步骤、检查清单、最佳实践	“排查 TypeScript 类型错误三步法”
<code>episodes/</code>	某次具体问题的完整解决过程	“2025-07 部署失败排查实录”
<code>principles/</code>	跨项目/跨技术的通用原则	“优先显式类型而非 <code>any</code> ”

4.4.3 示例：从 Markdown 仓库批量导入

假设你有一个 `/Notes/` 目录，里面是一堆 Markdown 技术笔记：

```
# 1. 创建提案池
mkdir -p ~/Dev/evo-kernel/ops/proposals/import-2026-07-24

# 2. 用脚本预处理（这里仅示意，实际根据你的笔记结构写）
python3 scripts/import-notes.py \
  --source ~/Notes \
  --target ~/Dev/evo-kernel/ops/proposals/import-2026-07-24 \
  --format evo-kernel
```

预处理后的一个提案文件示例：

```
---
id: ts-type-error-three-steps
name: TypeScript 类型错误排查三步法
type: playbook
status: candidate
scope: [debugging]
domains: [typescript]
triggers:
  - TypeScript 类型错误
  - type error
  - 类型排查
evidence:
  helpful: 0
  harmful: 0
verified_by: human
---
```

遇到 TypeScript 类型错误时，按以下顺序排查：

1. 先看错误位置的上下文，确认是「声明错误」还是「使用错误」；
2. 检查最近的类型断言、泛型参数、`as` 和 `satisfies`；
3. 若仍无法定位，临时用 `// @ts-expect-error` 标注并附带原因，而不是 `any`。

```
# 3. 人审并 curate (逐条或分批)
cd ~/Dev/evo-kernel
./bin/evo curate --file ops/proposals/import-2026-07-24/ts-type-error-three-steps.md --to playbook
```

4.4.4 研究与借鉴

Evo-Kernel 的导入模型参考了两种成熟方案：

- **Zettelkasten**: 强调 “fleeting note → literature note → permanent note” 的多阶段处理。临时捕获的笔记应在 24-48 小时内处理，大部分被丢弃，只有经过自己语言重写的原子思想才进入永久库。
- **RAG ingestion pipeline**: 典型的生产级流程包括 extraction → cleaning → chunking → metadata enrichment → embedding → indexing。其中最影响检索

质量的不是 chunk 大小，而是 chunk 的语义完整性、上下文线索和元数据治理。

这两种方案的共同点是：**不直接吞入原始资料**，而是经过一个「消化层」——Zettelkasten 靠人工重写，RAG 靠机器分段 + 元数据 + 人工校验。Evo-Kernel 的 `ops/proposals/` + `evo curate` 就是这个消化层。

提示

如果资料量很大，可以先用 LLM 批量生成提案草稿，但 `evo curate` 这一步不要批量自动执行。把 `curate` 当作“最终发布按钮”，保证进入注入通道的每一条都经过人眼确认。

4.5 工作流五：硬约束硬化

当某条经验涉及「会造成实际损失的危险操作」，且 skill/提示层已失效时，可以升为 hook。

```
./bin/evo solidify --id no-rm-rf-root --to hook \  
--matcher "rm\\s+-rf\\s+/" \  
--match-on command \  
--mode warn \  
--message "检测到针对根目录的 rm -rf，请人工确认" \  
--criteria-confirmed
```

红线

hook 准入四条件（缺一不可）：

1. 可程序化判定：matcher 写得出来、无歧义；
2. 违反代价高昂：不是风格问题，是会造成实际损失的操作；
3. 软层已失效：skill/提示层存在但仍被反复违反（有 harmful 记录佐证）；
4. matcher 覆盖率经复核：明列覆盖/未覆盖的形态与误报语境。

新 hook 必须先 `warn` 运行一个 `reflect` 周期，评估误报率后才能升 `block`。

5 周期维护

5.1 每周/每两周做一次

```
cd ~/Dev/evo-kernel
```

```
./bin/evo index rebuild    # 如果 curate/solidify 已经自动 rebuild
                           , 可跳过
./bin/evo audit            # 检查治理问题
./bin/evo reflect --save  # 生成周期报告
```

5.2 对账与计数

helpful/harmful 只能由离线对账单点写（不变量 I4）。主要通道是：

- **自动**: reflect/distill 过程中, Reflector 对每个 session 的注入清单判定四态 (adopted / relevant-unused / irrelevant / misleading), 写入 ops/log/reconcile.jsonl。
- **人工兜底**: 如果你明确知道某条经验被采用或误导了, 可以手动标记:

```
./bin/evo adopt --ids avoid-silent-bypass      # helpful +1
./bin/evo reject --ids outdated-command-pattern # harmful +1
```

之后需要 evo index rebuild 把计数汇总进 manifest。

5.3 治理动作

audit 发现的问题通常对应以下动作：

表 3: audit findings 与对应动作

级别	含义	动作
HIGH	harmful > helpful	evo reject --ids <id> 或 evo demote --id <id> --to archive
MID	superseded 但未归档	evo demote --id <id> --to archive
MID	lessons 中 status 非 candidate	迁出到正确 zone
MID	lessons 条目超过 30 天	决定 curate 或 archive
LOW	playbook / facts / principles 超过 90 天	复验内容是否仍有效

6 硬约束执行原理

6.1 guard 与 hook-guard

guard 是 Pi 的同步调用路径；hook-guard 是 Claude Code PreToolUse hook 的封装。二者共用 ops/constraints/*.json 中的规则。

图 4 展示了 guard 的决策流程。

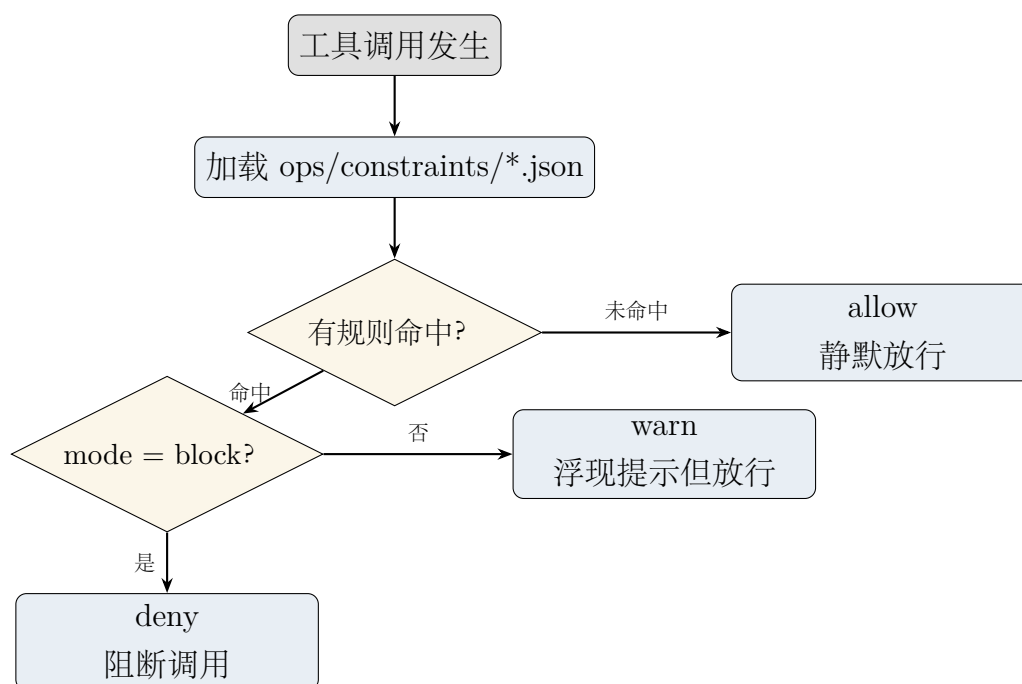


图 4: guard 决策流程

注意

fail-open 原则： guard 自身异常（约束文件损坏、正则非法等）时，一律按 allow 处理。只有「规则正确匹配且 mode=block」才会 deny。这是为了保证经验治理层不会反过来阻塞正常工作。

7 备份与恢复

7.1 备份机制

Evo-Kernel 有两层备份：

1. **主备份：** 每次 `curate` / `solidify` / `demote` 自动 `git push` 到私有 remote（RPO = 一次提交）。
2. **兜底备份：** 定期生成 `git bundle` 到本地异地介质：

```
cd ~/Dev/evo-kernel
git bundle create ~/evo-kernel-$(date +%Y%m%d).bundle --all
```

红线

原始日志不备份：ops/log/*.jsonl 含 prompt 明文，已 gitignore，永不入 remote/bundle。盘损时原始判据数据会丢失，只有 reflect 聚合后的历史保留。这是 §4.4 的诚实权衡。

7.2 恢复演练

每季度做一次：

```
TMPDIR=$(mktemp -d)
cd "$TMPDIR"
git clone git@github.com:zodyne/evo-kernel.git
cd evo-kernel
npm install
./bin/evo doctor      # 此时 harness 挂线项会 WARN/FAIL，属预期
./test/smoke.sh      # 必须 PASS=85 FAIL=0
rm -rf "$TMPDIR"
```

8 红线与注意事项

红线

1. **remote 必须私有**。经验条目含工作语境，公开 remote 直接违反 §4.3 红线。
2. **inbox / lessons 永不注入**。只有 curate 后的分区才有资格进入 recall 通道（不变量 I2）。
3. **helpful/harmful 只由 reconcile 单点写**。禁止实时反馈回路（不变量 I4）。
4. **curate 必须人审**。这是唯一入库口，负脱敏与指令样审查责任（不变量 I3）。
5. **原始日志不入 git**。ops/log/*.jsonl 全部 gitignore（不变量 I4 / §4.4）。
6. **hook 升 block 前必须过 warn 观察期**。且需人工复核 matcher 覆盖率 (§8)。
7. **不要多机并写**。remote 仅备份，第二台机器只读消费；多机并写会 ID 碰撞、计数合并未定义 (§4.1)。

8.1 常见误区

- 「我把文件拖进 `playbook/` 就行了吧？」不行。`curate` 是唯一能写入 `playbook/-facts/episodes/principles` 的入口。
- 「`recall` 空命中很多，是不是该上 `M1 FTS5`？」`M1` 判据是「注入精度连续 2 周期 $< 50\%$ 且 `recall` ≥ 200 次」。截至 2026-07-29 `recall` 累计 193 次、召回精度 54%，两条都未命中；且对账覆盖率仅 20%，精度本身还不可解读。做了也是过早优化。
- 「`transcript` 哨兵比高，是不是 `bug`？」不是 `bug`。`Claude Code` 会按策略清理 `transcript` 文件，登记与蒸馏之间存在腐烂窗口。由 `reflect` 周期跟踪，不是靠实施解决。
- 「我想在另一台机器上也 `curate`。」暂时不要。多机并写未设计，只读消费可以。

9 实施进度评估（截至 2026-07-29）

提示

本节所有数字取自 2026-07-29 当日在本机实跑的 `evo doctor --full` 与 `evo reflect` 输出，非估算。复现方式：`cd ~/Dev/evo-kernel && ./bin/evo doctor --full && ./bin/evo reflect`。

本节数字的时效性本身就是一条经验（条目 `doc-selfreported-counts-drift`）：初稿按 07-28 数据写成，隔一天复核时库规模已从 50 涨到 67、`smoke` 从 85 涨到 93、待审提案从 11 涨到 22。**** 读到本节请先按上面的命令复跑一遍 ****，不要直接引用纸面数字。

9.1 总体结论

内核已完整落地并进入日常生产使用：命令面 26 个、`smoke` 93 项断言全绿、双 `harness` 挂线完好、飞轮（`capture` $\rightarrow$ `recall` $\rightarrow$ 蒸馏 $\rightarrow$ `curate`）已闭环并自动化运行。设计文档 `design/final-architecture.md` §9.3 列出的 4 项「现在就做」（无判据）清单已全部核销。

Phase 1-3（`FTS5` / 向量 / `LightRAG`）按设计全部处于判据门控状态，当前无一命中——这是符合演进纪律的预期状态，不是欠账。

真正的短板不在实现，在 **** 治理链路的下游 ****：固化梯度（`solidify`）与退役出口（`demote`）代码就绪但零生产实例，硬约束 2 条规则全部停在 `warn` 且误报率 55-71%，离线对账覆盖率仅 20%（低于 30% 的可解读门槛）。

9.2 Phase 0 清单核销

表 4: design/final-architecture.md §9.3 「现在就做」四项		
项	状态	证据
M0 解析器修复	已完成	引入 js-yaml，且显式用 CORE_SCHEMA 堵住日期被静默转 Date 的坑 (commit ed0c80e)
K0a primer	已完成	evo primer --install; doctor 项 17 PASS: 两处已装, review_after 2026-09-25
去锚定 L1	已完成	primer 常驻块落 CLAUDE.md/AGENTS.md 为主、kernel 注入为辅 (commit 312e62c)
日常飞轮	运行中	recall 累计 193 次; 库从 18 条增至 67 条; 蒸馏已自动化

9.3 能力落地矩阵

表 5: 七条主链路的实现与验证状态		
能力	状态	实测证据
捕获 capture	生产可用	inbox 零判断入口在用；积压已按「junk drawer 前兆」清理过一轮 (420ec73)
检索注入 recall	生产可用	193 次调用，76% 有注入命中 (147/193)；后端仍为 scan (Phase 1 未触发)
离线蒸馏 distill	已自动化	SessionEnd 触发器 + launchd 每日 03:30 兜底 (--max 3、带锁)；queue / mark-distilled / catalog 三命令支撑队列与查重
人审入库 curate	生产可用	51 条 validated 已入库；入 validated 区改写 status 的状态机漏洞已修 (cc45bf6)；未跟踪提案致 git add 整批 fatal、commit 从未成功的缺陷已修 (cc3ff21)
离线对账 reconcile	部分验证	39 例已判四态 (adopted 6 / relevant-unused 15 / irrelevant 18 / misleading 0)，覆盖率仅 20%
技能固化 solidify	路径未验证	代码与 smoke 就绪，但生产实例为 0；现有 4 个 SKILL 均为手写运维技能，非条目固化产物；reflect 已挂出 6 条固化候选待处理
硬约束 guard	路径未验证	2 条规则，均停在 warn: dangerous-rm-rf 误报率 71% (59 裸命中/42 引号内提及) 、evo-curate-needs-human 误报率 55% 且 criteria_confirmed=false, 双双未达 §8 准入第 4 条

9.4 库存与飞轮实测数据

表 6: 2026-07-29 实测规模		
指标	当前值	备注
条目总数	67	其中可注入 51 条 (validated)
分区分布	playbook 41 / lessons 9 / inbox 7 / facts 5 / episodes 4 / principles 1	principles 单薄是真实缺口
待审提案	22	一天内从 11 翻倍，后台蒸馏产出快于人审
ops/archive/skills	0 4	退役出口从未被走过 均为手写运维技能
constraints	2 (均 warn)	无 block 规则
CLI 命令面	26	设计资产表记录的 617 行已增至 1463 行
smoke 断言	PASS=93 FAIL=0	手册旧版记录为 61
doctor	16 PASS / 1 WARN / 0 FAIL	WARN = 工作区有未提交变更
recall 累计	193 次	空命中任务 46 个
transcript 哨兵比	36/81 (44%)	已由 FAIL/WARN 转为 PASS

9.5 判据门控状态

表 7: Phase 1-3 判据（全部未命中，符合预期）			
判据	阈值	当前值	结论
M1 recall 累计	≥ 200	193	未命中(已逼近)
M1 召回精度（检索层）	连续 2 周期 < 50%	54% (21/39)	未命中
采纳率（应用层）	观察项，阈值待数据	29% (6/21)	观察
M1 对账覆盖率	≥ 30% 方可解读	20% (39/193)	不足，精度暂不可解读
M2 条目数（向量）	≥ 500	67	未命中
P4 多跳关系需求	空命中呈关系特征	46 个空命中，多为闲聊/短指令	未命中

注意

对账覆盖率 20% 是当前最关键的观测缺口：精度建立在 193 次注入里仅 39 次被判定的基础上，样本偏斜风险高。在覆盖率提到 30% 以前，不应据此讨论换检索后端。

另需注意精度口径已拆分为两层：**召回精度 54%**（检索层——找出来的条目是否相关）与 **采纳率 29%**（应用层——相关条目是否真被用上）。两者差距说明当前瓶颈更可能在「注入了但没被采纳」，而不在检索本身——这进一步支持「先修蒸馏与采纳纪律，而非先换检索后端」。

9.6 尚未验证的路径（诚实清单）

1. **固化梯度是空转的。** `solidify --to skill` 从未在生产跑过一次，`ops/archive/` 为空。这意味着「经验 → 技能」和「经验退役」这两条设计承诺的出口，只有 `smoke` 覆盖、没有真实使用反馈。`reflect` 当前挂着 6 条固化候选（最高的是 `verify-external-references`, `helpful=5`）与 1 条退役候选，是消除该空白的现成入口。
2. **硬约束层实质未启用。** 2 条规则全部停在 `warn: matcher` 是子串匹配，分不清「执行」与「引号内提及」，误报率分别为 71% 与 55%，`reflect` 对两条都明确不建议升 `block`。硬化梯度目前仍是纸面能力。
3. **principles 仅 1 条。** 跨域普适原则本应是最高价值层，实际产出集中在 `playbook` (41 条)。蒸馏产物的层级分布偏低。
4. **提案积压 22 条。** 人审是设计上的瓶颈也是必要环节，但积压量一天内从 11 翻倍到 22——后台蒸馏的产出速度已明显快于人审速度，「消化层反成堆积层」不再是风险而是正在发生。
5. **多机并写仍未设计。** `remote` 只作备份，第二台机器只读消费的约束未变。

9.7 建议的下一步（按优先级）

1. 走通一次完整的 `solidify --to skill`，用 `reflect` 挂出的高 `helpful` 候选打通固化出口；同步走一次 `demote` 清掉空转条目。
2. 提升对账覆盖率至 30% 以上——修蒸馏与采纳纪律优先于修检索（采纳率 29% 明显低于召回精度 54%）。
3. 收窄两条约束的 `matcher`（剥离引号语境，区分执行与提及），再谈 `warn` → `block`。
4. 消化 22 条待审提案，并给后台蒸馏加产出节流——否则人审永远追不上。
5. Phase 1-3 继续保持门控，不提前动工。

10 命令速查表

表 8: Evo-Kernel 26 命令速查

命令	用途	典型场景
<code>capture "..."</code>	零判断捕获	随时记录想法
<code>recall --task "..."</code>	主动检索	手动查经验
<code>hook-recall</code>	harness 注入入口	由 hooks 自动调用
<code>session-end</code>	手动登记会话	hook 未触发时
<code>hook-session-end</code>	自动登记入口	由 hooks 自动调用
<code>adopt --ids</code>	人工 +helpful	明确知道某条被采用
<code>reject --ids</code>	人工 +harmful	明确知道某条误导
<code>reconcile --ids --state</code>	Reflector 四态对账	蒸馏时自动写
<code>queue</code>	待蒸馏队列 TSV	后台驱动器输入
<code>mark-distilled --ids</code>	回写蒸馏完成	蒸馏收尾
<code>curate --file --to</code>	人审入库	核心流程
<code>slice --session</code>	transcript 切片	蒸馏前准备
<code>index rebuild</code>	重建 manifest	curate 后自动做
<code>primer [--install]</code>	常驻背景块	装到 CLAUDE.md / AGENTS.md
<code>candidates</code>	列出候选条目	agentic 粗筛
<code>catalog</code>	id×triggers 查重清单	Reflector 落笔前查重
<code>get --ids</code>	取全文	agentic 精筛
<code>reflect --save</code>	周期复盘	每周/每两周
<code>audit</code>	治理检查	配合 reflect
<code>inbox</code>	待处理清单	查看 capture 和 refs
<code>solidify --id --to skill</code>	技能固化	经验变 SKILL.md
<code>solidify --id --to hook</code>	硬约束注册	危险操作防护
<code>demote --id --to archive</code>	退役	经验过期/有害
<code>guard / hook-guard</code>	约束执行	由 harness 调用
<code>link</code>	同步 skills 软链	solidify 后自动做
<code>doctor [--full]</code>	部署自检	每日/每次重大变更后